

processor you use to run this code, it can probably look only a few moves forward without taking an unreasonable amount of time (a few minutes or more) for each decision. It's natural to want to enhance our AI to make it better.

One thing we'll definitely want to improve is our AI's speed. It's slow because of the large size of the game trees it has to work through. One of the main ways to improve the performance of minimax is by pruning the game tree. Pruning, as you might remember from Chapter 9, is exactly what it sounds like: we remove branches from the tree if we consider them exceptionally poor or if they represent a duplicate of another branch. Pruning is not trivial to implement and requires learning yet more algorithms to do it well. One example is the *alpha-beta pruning algorithm*, which will stop checking particular sub-branches if they are certainly worse than sub-branches elsewhere in the tree.

Another natural improvement to our AI would be to enable it to work with different rules or different games. For example, a commonly used rule in dots and boxes is that after earning a point, a player gets to draw another line. Sometimes this results in a cascade, in which one player completes many boxes in a row in a single turn. This simple change, which was called "make it, take it" on my elementary school playground, changes the game's strategic considerations and will require some changes to our code. You can also try to implement an AI that plays dots and boxes on a lattice that has a cross shape or some other exotic shape that could influence strategy. The beauty of minimax is that it doesn't require subtle strategic understanding; it requires only an ability to look ahead, and that's why a coder who isn't good at chess can write an implementation of minimax that can beat them at chess.

There are some powerful methods that go beyond the scope of this chapter that can improve the performance of computer AI. These methods include reinforcement learning (where a chess program, for example, plays against itself to get better), Monte Carlo methods (where a shogi program generates random future shogi games to help understand possibilities), and neural networks (where a tic-tac-toe program predicts what its opponent will do using a machine learning method similar to what we discussed in Chapter 9). These methods are powerful and remarkable, but they mostly